

# Ejemplo reproducible: creación de grafos desde datos tabulares y entrenamiento de una GNN

Dataset cardiovascular de Kaggle + PyTorch Geometric

Documento generado con asistencia de IA

2026-07-03

## Índice

|          |                                                                           |           |
|----------|---------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Propósito del documento</b>                                            | <b>2</b>  |
| <b>2</b> | <b>Idea general</b>                                                       | <b>2</b>  |
| <b>3</b> | <b>Justificación conceptual</b>                                           | <b>2</b>  |
| <b>4</b> | <b>Requisitos de instalación</b>                                          | <b>2</b>  |
| 4.1      | Crear el proyecto . . . . .                                               | 3         |
| 4.2      | Instalar PyTorch con CUDA . . . . .                                       | 3         |
| 4.3      | Instalar las demás dependencias . . . . .                                 | 3         |
| 4.4      | Autenticación en Kaggle . . . . .                                         | 3         |
| <b>5</b> | <b>Flujo completo del pipeline</b>                                        | <b>3</b>  |
| <b>6</b> | <b>Código explicado por chunks</b>                                        | <b>3</b>  |
| 6.1      | Chunk 1. Importación de librerías . . . . .                               | 4         |
| 6.1.1    | Explicación . . . . .                                                     | 4         |
| 6.2      | Chunk 2. Parámetros generales del experimento . . . . .                   | 4         |
| 6.2.1    | Explicación . . . . .                                                     | 4         |
| 6.3      | Chunk 3. Fijar la semilla aleatoria . . . . .                             | 4         |
| 6.3.1    | Explicación . . . . .                                                     | 5         |
| 6.4      | Chunk 4. Buscar el archivo CSV dentro del dataset descargado . . . . .    | 5         |
| 6.4.1    | Explicación . . . . .                                                     | 5         |
| 6.5      | Chunk 5. Descargar y cargar el dataset tabular . . . . .                  | 5         |
| 6.5.1    | Explicación . . . . .                                                     | 5         |
| 6.6      | Chunk 6. Limpieza básica del dataset . . . . .                            | 5         |
| 6.6.1    | Explicación . . . . .                                                     | 6         |
| 6.7      | Chunk 7. Construcción del grafo kNN . . . . .                             | 6         |
| 6.7.1    | Explicación . . . . .                                                     | 7         |
| 6.8      | Chunk 8. Crear el objeto <code>Data</code> de PyTorch Geometric . . . . . | 8         |
| 6.8.1    | Explicación . . . . .                                                     | 9         |
| 6.9      | Chunk 9. Definición del modelo GCN . . . . .                              | 9         |
| 6.9.1    | Explicación . . . . .                                                     | 10        |
| 6.10     | Chunk 10. Función de evaluación . . . . .                                 | 10        |
| 6.10.1   | Explicación . . . . .                                                     | 10        |
| 6.11     | Chunk 11. Función principal de entrenamiento . . . . .                    | 11        |
| 6.11.1   | Explicación . . . . .                                                     | 12        |
| 6.12     | Chunk 12. Punto de entrada del programa . . . . .                         | 13        |
| 6.12.1   | Explicación . . . . .                                                     | 13        |
| <b>7</b> | <b>Script completo consolidado</b>                                        | <b>13</b> |
| <b>8</b> | <b>Ejecución</b>                                                          | <b>18</b> |

|                                                               |    |
|---------------------------------------------------------------|----|
| 9 Ajustes para una GPU de 8 GB                                | 19 |
| 10 Advertencia metodológica                                   | 19 |
| 11 Interpretación biomédica del ejemplo                       | 19 |
| 12 Cómo trasladar este esquema a proyectos biomédicos propios | 19 |
| 13 Conclusión                                                 | 20 |
| 14 Referencias breves                                         | 20 |

## 1 Propósito del documento

Este documento presenta un ejemplo reproducible para convertir un **dataset tabular** en un **grafo** y entrenar una **Graph Neural Network (GNN)** usando PyTorch Geometric.

El ejemplo usa el dataset **Cardiovascular Disease dataset** de Kaggle, disponible en:

<https://www.kaggle.com/datasets/sulianova/cardiovascular-disease-dataset>

La página del dataset fue verificada antes de preparar este documento. Kaggle describe el dataset como un conjunto con **70.000 registros de pacientes**, **11 características** y una variable objetivo.

La tarea predictiva será clasificar si un paciente presenta o no enfermedad cardiovascular.

## 2 Idea general

Partimos de una tabla convencional:

```
paciente | edad | peso | presión | colesterol | glucosa | fumador | cardio
```

La convertimos en un grafo:

```
Nodo    = paciente
Arista  = relación de similitud clínica entre pacientes
x       = variables clínicas del paciente
y       = etiqueta: presencia o ausencia de enfermedad cardiovascular
```

La regla de construcción del grafo será:

Dos pacientes se conectan si son clínicamente parecidos.

Para medir esa similitud se usará **k vecinos más cercanos** (*k-nearest neighbors*, kNN). Así, cada paciente se conecta con sus K\_NEIGHBORS pacientes más similares.

## 3 Justificación conceptual

Un modelo tabular clásico aprende así:

```
Paciente -> variables clínicas -> predicción
```

Una GNN aprende así:

```
Paciente -> variables clínicas
          -> pacientes vecinos similares
          -> estructura global del grafo
          -> predicción
```

La diferencia central es que la GNN no usa cada fila de forma aislada. También usa la información que fluye desde los vecinos del nodo.

## 4 Requisitos de instalación

Este ejemplo está pensado para una GPU NVIDIA de **8 GB**, aunque también puede ejecutarse en CPU usando menos nodos.

## 4.1 Crear el proyecto

```
mkdir gnn-tabular-cardiovascular
cd gnn-tabular-cardiovascular
```

```
uv venv --python 3.12
```

Activar el entorno en Linux o WSL:

```
source .venv/bin/activate
```

Activar el entorno en Windows PowerShell:

```
.venv\Scripts\Activate.ps1
```

## 4.2 Instalar PyTorch con CUDA

Para una GPU NVIDIA reciente se puede usar, por ejemplo, CUDA 12.8:

```
uv pip install torch torchvision torchaudio --index-url
↪ https://download.pytorch.org/whl/cu128
```

Si tu versión de CUDA o driver es diferente, conviene revisar el selector oficial de instalación de PyTorch:

<https://pytorch.org/get-started/locally/>

## 4.3 Instalar las demás dependencias

```
uv pip install torch-geometric pandas numpy scikit-learn kagglehub
```

## 4.4 Autenticación en Kaggle

```
uv run python -c "import kagglehub; kagglehub.login()"
```

No se deben pegar tokens de Kaggle en repositorios ni en chats.

# 5 Flujo completo del pipeline

El flujo general será:

```
Dataset tabular de Kaggle
↓
DataFrame de pandas
↓
Limpieza básica
↓
Normalización de variables
↓
Construcción del grafo por k vecinos más cercanos
↓
Objeto Data de PyTorch Geometric
↓
Modelo GCN
↓
Entrenamiento con nodos etiquetados
↓
Evaluación en validación y prueba
```

# 6 Código explicado por chunks

Los siguientes chunks componen un único archivo llamado:

```
train_tabular_as_graph.py
```

Cada chunk puede leerse como una sección del script o como una celda de notebook.

## 6.1 Chunk 1. Importación de librerías

```
from pathlib import Path

import kagglehub
import numpy as np
import pandas as pd
import torch
import torch.nn.functional as F

from sklearn.model_selection import train_test_split
from sklearn.neighbors import NearestNeighbors
from sklearn.preprocessing import StandardScaler

from torch_geometric.data import Data
from torch_geometric.nn import GCNConv
```

### 6.1.1 Explicación

kagglehub permite descargar el dataset desde Kaggle. pandas y numpy se usan para manipular datos tabulares. torch es la base para entrenar redes neuronales y torch\_geometric es la librería especializada para grafos y GNN.

De sklearn se usan tres herramientas:

```
train_test_split    -> dividir datos en entrenamiento, validación y prueba
NearestNeighbors    -> construir el grafo por similitud entre pacientes
StandardScaler      -> normalizar las variables numéricas
```

La clase Data de PyTorch Geometric representa un grafo homogéneo y puede contener atributos de nodos, aristas y etiquetas. En PyG, normalmente se usan atributos como data.x, data.edge\_index y data.y.

## 6.2 Chunk 2. Parámetros generales del experimento

```
DATASET_SLUG = "sulianova/cardiovascular-disease-dataset"
SEED = 42

# Para GPU de 8 GB, puedes empezar con 30.000 nodos.
# Si corre bien, sube a 50_000 o None para usar todo el dataset.
MAX_NODES = 30_000

# Número de vecinos por paciente.
K_NEIGHBORS = 10
```

### 6.2.1 Explicación

DATASET\_SLUG identifica el dataset dentro de Kaggle.

SEED ayuda a que la división de datos y la inicialización del modelo sean más reproducibles.

MAX\_NODES limita el número de pacientes usados. Como cada paciente será un nodo, esta variable controla el tamaño del grafo.

K\_NEIGHBORS indica cuántos vecinos tendrá cada paciente. Si K\_NEIGHBORS = 10, cada paciente se conecta con los 10 pacientes clínicamente más similares.

A mayor K\_NEIGHBORS, mayor número de aristas y mayor consumo de memoria.

## 6.3 Chunk 3. Fijar la semilla aleatoria

```
def set_seed(seed: int = 42) -> None:
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
```

### 6.3.1 Explicación

Esta función controla parte de la aleatoriedad del experimento.

En redes neuronales hay aleatoriedad en la inicialización de pesos, la división de datos y operaciones como dropout. Fijar una semilla no garantiza reproducibilidad perfecta en GPU, pero mejora la estabilidad de los resultados.

## 6.4 Chunk 4. Buscar el archivo CSV dentro del dataset descargado

```
def find_csv(root: Path) -> Path:
    csv_files = list(root.rglob("*.csv"))
    if not csv_files:
        raise FileNotFoundError(f"No se encontró ningún CSV dentro de {root}")
    return csv_files[0]
```

### 6.4.1 Explicación

Cuando se descarga un dataset de Kaggle, el archivo puede quedar en una subcarpeta. Esta función busca recursivamente archivos .csv dentro de la carpeta descargada.

Si no encuentra ningún CSV, lanza un error explícito. Si encuentra uno o más, toma el primero.

## 6.5 Chunk 5. Descargar y cargar el dataset tabular

```
def load_tabular_dataset() -> pd.DataFrame:
    dataset_path = Path(kagglehub.dataset_download(DATASET_SLUG))
    csv_path = find_csv(dataset_path)

    print(f"Dataset descargado en: {dataset_path}")
    print(f"Archivo CSV usado: {csv_path}")

    # Este dataset suele venir separado por punto y coma.
    df = pd.read_csv(csv_path, sep=";")

    print("\nColumnas originales:")
    print(df.columns.tolist())

    return df
```

### 6.5.1 Explicación

Esta función descarga el dataset desde Kaggle y lo carga como un DataFrame.

La línea:

```
dataset_path = Path(kagglehub.dataset_download(DATASET_SLUG))
```

descarga el dataset.

Luego se localiza el archivo CSV:

```
csv_path = find_csv(dataset_path)
```

Después se carga con pandas:

```
df = pd.read_csv(csv_path, sep=";")
```

Este dataset suele estar separado por punto y coma, por eso se usa `sep=";"`.

## 6.6 Chunk 6. Limpieza básica del dataset

```
def clean_dataframe(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()

    # Eliminar identificador si existe.
    if "id" in df.columns:
```

```

df = df.drop(columns=["id"])

# Limpieza básica de valores fisiológicamente extremos.
# No es una limpieza clínica definitiva; es solo para evitar outliers groseros.
df = df[
    (df["height"].between(120, 220))
    & (df["weight"].between(35, 200))
    & (df["ap_hi"].between(80, 240))
    & (df["ap_lo"].between(40, 160))
    & (df["ap_hi"] >= df["ap_lo"])
]

# Mezclar filas para evitar sesgos por orden.
df = df.sample(frac=1.0, random_state=SEED).reset_index(drop=True)

if MAX_NODES is not None:
    df = df.iloc[:MAX_NODES].copy()

return df

```

### 6.6.1 Explicación

Primero se elimina la columna `id`, porque identifica al paciente pero no debe ser una variable predictora.

Después se filtran valores fisiológicamente extremos. Esta limpieza no sustituye un protocolo clínico riguroso; solo evita que errores muy grandes distorsionen la construcción del grafo.

También se exige:

```
df["ap_hi"] >= df["ap_lo"]
```

porque la presión sistólica debe ser mayor o igual que la diastólica.

Finalmente, se mezclan las filas y se limita el tamaño del dataset mediante `MAX_NODES`.

## 6.7 Chunk 7. Construcción del grafo kNN

```

def build_knn_graph(x_np: np.ndarray, k: int) -> torch.Tensor:
    """
    Construye edge_index usando k vecinos más cercanos.

    x_np:
        matriz numpy [num_pacientes, num_variables]

    Retorna:
        edge_index con forma [2, num_aristas]
    """

    print(f"\nConstruyendo grafo kNN con k={k}...")

    nbrs = NearestNeighbors(
        n_neighbors=k + 1,
        metric="euclidean",
        algorithm="auto",
        n_jobs=-1,
    )

    nbrs.fit(x_np)

    # indices tiene forma [num_nodos, k+1]
    # El primer vecino de cada punto suele ser el mismo punto.
    _, indices = nbrs.kneighbors(x_np)

    source_nodes = []

```

```

target_nodes = []

num_nodes = x_np.shape[0]

for i in range(num_nodes):
    neighbors = indices[i, 1:] # quitar el propio nodo

    for j in neighbors:
        source_nodes.append(i)
        target_nodes.append(j)

    # Hacer el grafo no dirigido:
    source_nodes.append(j)
    target_nodes.append(i)

edge_index = torch.tensor(
    [source_nodes, target_nodes],
    dtype=torch.long,
)

return edge_index

```

### 6.7.1 Explicación

Este es el chunk central del documento: aquí la tabla se convierte en grafo.

La entrada `x_np` tiene forma:

```
[num_pacientes, num_variables]
```

Por ejemplo:

30000 pacientes x 11 variables clínicas

`NearestNeighbors` busca, para cada paciente, los pacientes más similares según distancia euclidiana:

```

nbrs = NearestNeighbors(
    n_neighbors=k + 1,
    metric="euclidean",
)

```

Se usa `k + 1` porque el vecino más cercano de un paciente suele ser el propio paciente. Luego se elimina con:

```
neighbors = indices[i, 1:]
```

Para cada paciente `i` y vecino `j`, se crea una arista:

Paciente `i` -> Paciente `j`

También se agrega la conexión inversa:

Paciente `j` -> Paciente `i`

Así el grafo se trata como no dirigido:

Paciente `i` --- Paciente `j`

PyTorch Geometric representa las aristas con `edge_index`:

```

edge_index[0] = nodos origen
edge_index[1] = nodos destino

```

Ejemplo simplificado:

```

edge_index =
[
    [0, 0, 1, 2],
    [1, 2, 0, 0]
]

```

Eso significa:

```
0 -> 1
0 -> 2
1 -> 0
2 -> 0
```

## 6.8 Chunk 8. Crear el objeto Data de PyTorch Geometric

```
def create_pyg_data(df: pd.DataFrame) -> Data:
    target_col = "cardio"

    if target_col not in df.columns:
        raise ValueError(f"No se encontró la columna objetivo '{target_col}'.")

    y_np = df[target_col].values.astype(np.int64)

    feature_df = df.drop(columns=[target_col])

    scaler = StandardScaler()
    x_np = scaler.fit_transform(feature_df.values).astype(np.float32)

    edge_index = build_knn_graph(x_np, K_NEIGHBORS)

    x = torch.tensor(x_np, dtype=torch.float)
    y = torch.tensor(y_np, dtype=torch.long)

    num_nodes = x.shape[0]
    indices = np.arange(num_nodes)

    train_idx, temp_idx = train_test_split(
        indices,
        train_size=0.70,
        random_state=SEED,
        stratify=y_np,
    )

    val_idx, test_idx = train_test_split(
        temp_idx,
        train_size=0.50,
        random_state=SEED,
        stratify=y_np[temp_idx],
    )

    train_mask = torch.zeros(num_nodes, dtype=torch.bool)
    val_mask = torch.zeros(num_nodes, dtype=torch.bool)
    test_mask = torch.zeros(num_nodes, dtype=torch.bool)

    train_mask[train_idx] = True
    val_mask[val_idx] = True
    test_mask[test_idx] = True

    data = Data(
        x=x,
        edge_index=edge_index,
        y=y,
        train_mask=train_mask,
        val_mask=val_mask,
        test_mask=test_mask,
    )

    return data
```



### 6.8.1 Explicación

Este chunk convierte el `DataFrame` en un objeto `Data` de PyTorch Geometric.

La variable objetivo es:

```
target_col = "cardio"
```

Por tanto:

```
y = presencia o ausencia de enfermedad cardiovascular
```

Las variables predictoras son todas las columnas menos `cardio`:

```
feature_df = df.drop(columns=[target_col])
```

Después se normalizan las variables:

```
scaler = StandardScaler()
x_np = scaler.fit_transform(feature_df.values).astype(np.float32)
```

Esto es importante porque variables como edad, peso, presión y glucosa pueden tener escalas distintas. La distancia euclidiana usada en kNN sería inadecuada si no se normalizan las variables.

Luego se construye el grafo:

```
edge_index = build_knn_graph(x_np, K_NEIGHBORS)
```

El objeto final contiene:

```
data.x          -> variables clínicas de los pacientes
data.edge_index -> conexiones entre pacientes similares
data.y          -> etiqueta cardiovascular
data.train_mask -> nodos usados para entrenamiento
data.val_mask   -> nodos usados para validación
data.test_mask  -> nodos usados para prueba
```

La división usada es:

```
70% entrenamiento
15% validación
15% prueba
```

## 6.9 Chunk 9. Definición del modelo GCN

```
class TabularGCN(torch.nn.Module):
    def __init__(self, in_channels: int, hidden_channels: int, out_channels: int):
        super().__init__()

        self.conv1 = GCNConv(in_channels, hidden_channels)
        self.conv2 = GCNConv(hidden_channels, hidden_channels)
        self.classifier = torch.nn.Linear(hidden_channels, out_channels)

    def forward(self, data: Data) -> torch.Tensor:
        x, edge_index = data.x, data.edge_index

        x = self.conv1(x, edge_index)
        x = F.relu(x)
        x = F.dropout(x, p=0.35, training=self.training)

        x = self.conv2(x, edge_index)
        x = F.relu(x)

        x = self.classifier(x)

        return x
```

### 6.9.1 Explicación

Este chunk define una red neuronal sobre grafos. El modelo usa dos capas GCNConv y una capa final de clasificación.

GCNConv es una implementación del operador de convolución de grafos asociado al modelo GCN. En términos simples, permite que cada nodo actualice su representación usando información de sus vecinos.

La primera capa recibe las variables clínicas originales:

```
in_channels = número de variables clínicas
```

y produce una representación interna:

```
hidden_channels = 64
```

La segunda capa vuelve a propagar información entre vecinos.

La capa final:

```
self.classifier = torch.nn.Linear(hidden_channels, out_channels)
```

produce la predicción. Como el problema es binario:

```
out_channels = 2
```

La línea esencial es:

```
x = self.conv1(x, edge_index)
```

Esto significa:

Cada paciente actualiza su representación usando sus propias variables y las variables de sus vecinos.

## 6.10 Chunk 10. Función de evaluación

```
@torch.no_grad()
def evaluate(model: torch.nn.Module, data: Data, mask: torch.Tensor) -> float:
    model.eval()

    logits = model(data)
    pred = logits[mask].argmax(dim=1)

    correct = (pred == data.y[mask]).sum().item()
    total = int(mask.sum())

    return correct / total
```

### 6.10.1 Explicación

Esta función calcula el accuracy.

Primero se pone el modelo en modo evaluación:

```
model.eval()
```

Eso desactiva operaciones como dropout.

Luego se calculan las salidas del modelo:

```
logits = model(data)
```

logits contiene los puntajes para cada clase.

Después se seleccionan solo los nodos indicados por mask:

```
logits[mask]
```

La máscara puede ser train\_mask, val\_mask o test\_mask.

La clase predicha es la de mayor puntaje:

```
pred = logits[mask].argmax(dim=1)
```

Finalmente se calcula:

```
accuracy = predicciones correctas / total de nodos evaluados
```

El decorador `@torch.no_grad()` indica que no se necesitan gradientes durante la evaluación. Esto reduce consumo de memoria.

## 6.11 Chunk 11. Función principal de entrenamiento

```
def train() -> None:
    set_seed(SEED)

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    print("Dispositivo:", device)

    if device.type == "cuda":
        print("GPU:", torch.cuda.get_device_name(0))
        mem_gb = torch.cuda.get_device_properties(0).total_memory / 1024**3
        print(f"Memoria GPU: {mem_gb:.2f} GB")

    df = load_tabular_dataset()
    df = clean_dataframe(df)

    print("\nDimensiones después de limpieza:")
    print(df.shape)

    print("\nDistribución de la variable objetivo:")
    print(df["cardio"].value_counts(normalize=True).sort_index())

    data = create_pyg_data(df)

    print("\nResumen del grafo")
    print("-----")
    print(f"Nodos: {data.num_nodes}")
    print(f"Aristas: {data.num_edges}")
    print(f"Variables por nodo: {data.num_node_features}")
    print(f"Clases: {int(data.y.max().item()) + 1}")
    print(f"Nodos entrenamiento: {int(data.train_mask.sum())}")
    print(f"Nodos validación: {int(data.val_mask.sum())}")
    print(f"Nodos prueba: {int(data.test_mask.sum())}")

    data = data.to(device)

    model = TabularGCN(
        in_channels=data.num_node_features,
        hidden_channels=64,
        out_channels=2,
    ).to(device)

    optimizer = torch.optim.Adam(
        model.parameters(),
        lr=0.01,
        weight_decay=5e-4,
    )

    best_val_acc = 0.0
    best_state = None

    for epoch in range(1, 201):
```

```

model.train()
optimizer.zero_grad()

logits = model(data)
loss = F.cross_entropy(
    logits[data.train_mask],
    data.y[data.train_mask],
)

loss.backward()
optimizer.step()

train_acc = evaluate(model, data, data.train_mask)
val_acc = evaluate(model, data, data.val_mask)

if val_acc > best_val_acc:
    best_val_acc = val_acc
    best_state = {
        k: v.detach().cpu().clone()
        for k, v in model.state_dict().items()
    }

if epoch == 1 or epoch % 10 == 0:
    print(
        f"Epoch {epoch:03d} | "
        f"Loss: {loss.item():.4f} | "
        f"Train Acc: {train_acc:.4f} | "
        f"Val Acc: {val_acc:.4f}"
    )

if best_state is not None:
    model.load_state_dict(best_state)

test_acc = evaluate(model, data, data.test_mask)

print("\nResultado final")
print("-----")
print(f"Mejor accuracy validación: {best_val_acc:.4f}")
print(f"Accuracy test: {test_acc:.4f}")

```

### 6.11.1 Explicación

Esta función coordina el experimento completo.

Primero fija la semilla:

```
set_seed(SEED)
```

Luego detecta si hay GPU:

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

Después carga, limpia y transforma el dataset:

```
df = load_tabular_dataset()
df = clean_dataframe(df)
data = create_pyg_data(df)
```

La tabla queda convertida en un objeto Data.

Luego el grafo se mueve a GPU o CPU:

```
data = data.to(device)
```

Se crea el modelo:

```
model = TabularGCN(
    in_channels=data.num_node_features,
    hidden_channels=64,
    out_channels=2,
).to(device)
```

Se define el optimizador:

```
optimizer = torch.optim.Adam(
    model.parameters(),
    lr=0.01,
    weight_decay=5e-4,
)
```

El ciclo de entrenamiento se ejecuta durante 200 épocas:

```
for epoch in range(1, 201):
```

La pérdida se calcula solo sobre los nodos de entrenamiento:

```
loss = F.cross_entropy(
    logits[data.train_mask],
    data.y[data.train_mask],
)
```

Esto es importante: aunque el grafo contiene nodos de validación y prueba, sus etiquetas no se usan para actualizar los pesos.

Durante el entrenamiento se guarda el mejor modelo según el accuracy de validación. Al final se evalúa en el conjunto de prueba.

## 6.12 Chunk 12. Punto de entrada del programa

```
if __name__ == "__main__":
    train()
```

### 6.12.1 Explicación

Este bloque indica que, cuando se ejecute el archivo directamente, se llame la función `train()`.

Al ejecutar:

```
uv run python train_tabular_as_graph.py
```

Python ejecutará todo el flujo:

1. cargar dataset
2. limpiar tabla
3. construir grafo
4. crear GNN
5. entrenar
6. evaluar

## 7 Script completo consolidado

El siguiente script reúne todos los chunks anteriores.

```
from pathlib import Path

import kagglehub
import numpy as np
import pandas as pd
import torch
import torch.nn.functional as F

from sklearn.model_selection import train_test_split
from sklearn.neighbors import NearestNeighbors
```

```

from sklearn.preprocessing import StandardScaler

from torch_geometric.data import Data
from torch_geometric.nn import GCNConv

DATASET_SLUG = "sulianova/cardiovascular-disease-dataset"
SEED = 42
MAX_NODES = 30_000
K_NEIGHBORS = 10

def set_seed(seed: int = 42) -> None:
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

def find_csv(root: Path) -> Path:
    csv_files = list(root.rglob("*.csv"))
    if not csv_files:
        raise FileNotFoundError(f"No se encontró ningún CSV dentro de {root}")
    return csv_files[0]

def load_tabular_dataset() -> pd.DataFrame:
    dataset_path = Path(kagglehub.dataset_download(DATASET_SLUG))
    csv_path = find_csv(dataset_path)

    print(f"Dataset descargado en: {dataset_path}")
    print(f"Archivo CSV usado: {csv_path}")

    df = pd.read_csv(csv_path, sep=";")

    print("\nColumnas originales:")
    print(df.columns.tolist())

    return df

def clean_dataframe(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()

    if "id" in df.columns:
        df = df.drop(columns=["id"])

    df = df[
        (df["height"].between(120, 220))
        & (df["weight"].between(35, 200))
        & (df["ap_hi"].between(80, 240))
        & (df["ap_lo"].between(40, 160))
        & (df["ap_hi"] >= df["ap_lo"])
    ]

    df = df.sample(frac=1.0, random_state=SEED).reset_index(drop=True)

    if MAX_NODES is not None:
        df = df.iloc[:MAX_NODES].copy()

    return df

```

```

def build_knn_graph(x_np: np.ndarray, k: int) -> torch.Tensor:
    print(f"\nConstruyendo grafo kNN con k={k}...")

    nbrs = NearestNeighbors(
        n_neighbors=k + 1,
        metric="euclidean",
        algorithm="auto",
        n_jobs=-1,
    )

    nbrs.fit(x_np)
    _, indices = nbrs.kneighbors(x_np)

    source_nodes = []
    target_nodes = []

    num_nodes = x_np.shape[0]

    for i in range(num_nodes):
        neighbors = indices[i, 1:]

        for j in neighbors:
            source_nodes.append(i)
            target_nodes.append(j)

            source_nodes.append(j)
            target_nodes.append(i)

    edge_index = torch.tensor(
        [source_nodes, target_nodes],
        dtype=torch.long,
    )

    return edge_index

def create_pyg_data(df: pd.DataFrame) -> Data:
    target_col = "cardio"

    if target_col not in df.columns:
        raise ValueError(f"No se encontró la columna objetivo '{target_col}'.")

    y_np = df[target_col].values.astype(np.int64)
    feature_df = df.drop(columns=[target_col])

    scaler = StandardScaler()
    x_np = scaler.fit_transform(feature_df.values).astype(np.float32)

    edge_index = build_knn_graph(x_np, K_NEIGHBORS)

    x = torch.tensor(x_np, dtype=torch.float)
    y = torch.tensor(y_np, dtype=torch.long)

    num_nodes = x.shape[0]
    indices = np.arange(num_nodes)

    train_idx, temp_idx = train_test_split(
        indices,
        train_size=0.70,
        random_state=SEED,
    )

```

```

        stratify=y_np,
    )

    val_idx, test_idx = train_test_split(
        temp_idx,
        train_size=0.50,
        random_state=SEED,
        stratify=y_np[temp_idx],
    )

    train_mask = torch.zeros(num_nodes, dtype=torch.bool)
    val_mask = torch.zeros(num_nodes, dtype=torch.bool)
    test_mask = torch.zeros(num_nodes, dtype=torch.bool)

    train_mask[train_idx] = True
    val_mask[val_idx] = True
    test_mask[test_idx] = True

    data = Data(
        x=x,
        edge_index=edge_index,
        y=y,
        train_mask=train_mask,
        val_mask=val_mask,
        test_mask=test_mask,
    )

    return data


class TabularGCN(torch.nn.Module):
    def __init__(self, in_channels: int, hidden_channels: int, out_channels: int):
        super().__init__()

        self.conv1 = GCNConv(in_channels, hidden_channels)
        self.conv2 = GCNConv(hidden_channels, hidden_channels)
        self.classifier = torch.nn.Linear(hidden_channels, out_channels)

    def forward(self, data: Data) -> torch.Tensor:
        x, edge_index = data.x, data.edge_index

        x = self.conv1(x, edge_index)
        x = F.relu(x)
        x = F.dropout(x, p=0.35, training=self.training)

        x = self.conv2(x, edge_index)
        x = F.relu(x)

        x = self.classifier(x)

        return x


@torch.no_grad()
def evaluate(model: torch.nn.Module, data: Data, mask: torch.Tensor) -> float:
    model.eval()

    logits = model(data)
    pred = logits[mask].argmax(dim=1)

    correct = (pred == data.y[mask]).sum().item()

```



```

total = int(mask.sum())

return correct / total

def train() -> None:
    set_seed(SEED)

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    print("Dispositivo:", device)

    if device.type == "cuda":
        print("GPU:", torch.cuda.get_device_name(0))
        mem_gb = torch.cuda.get_device_properties(0).total_memory / 1024**3
        print(f"Memoria GPU: {mem_gb:.2f} GB")

    df = load_tabular_dataset()
    df = clean_dataframe(df)

    print("\nDimensiones después de limpieza:")
    print(df.shape)

    print("\nDistribución de la variable objetivo:")
    print(df["cardio"].value_counts(normalize=True).sort_index())

    data = create_pyg_data(df)

    print("\nResumen del grafo")
    print("-----")
    print(f"Nodos: {data.num_nodes}")
    print(f"Aristas: {data.num_edges}")
    print(f"Variables por nodo: {data.num_node_features}")
    print(f"Clases: {int(data.y.max().item()) + 1}")
    print(f"Nodos entrenamiento: {int(data.train_mask.sum())}")
    print(f"Nodos validación: {int(data.val_mask.sum())}")
    print(f"Nodos prueba: {int(data.test_mask.sum())}")

    data = data.to(device)

    model = TabularGCN(
        in_channels=data.num_node_features,
        hidden_channels=64,
        out_channels=2,
    ).to(device)

    optimizer = torch.optim.Adam(
        model.parameters(),
        lr=0.01,
        weight_decay=5e-4,
    )

    best_val_acc = 0.0
    best_state = None

    for epoch in range(1, 201):
        model.train()
        optimizer.zero_grad()

        logits = model(data)
        loss = F.cross_entropy(

```

```

        logits[data.train_mask],
        data.y[data.train_mask],
    )

    loss.backward()
    optimizer.step()

    train_acc = evaluate(model, data, data.train_mask)
    val_acc = evaluate(model, data, data.val_mask)

    if val_acc > best_val_acc:
        best_val_acc = val_acc
        best_state = {
            k: v.detach().cpu().clone()
            for k, v in model.state_dict().items()
        }

    if epoch == 1 or epoch % 10 == 0:
        print(
            f"Epoch {epoch:03d} | "
            f"Loss: {loss.item():.4f} | "
            f"Train Acc: {train_acc:.4f} | "
            f"Val Acc: {val_acc:.4f}"
        )

    if best_state is not None:
        model.load_state_dict(best_state)

    test_acc = evaluate(model, data, data.test_mask)

    print("\nResultado final")
    print("-----")
    print(f"Mejor accuracy validación: {best_val_acc:.4f}")
    print(f"Accuracy test: {test_acc:.4f}")

if __name__ == "__main__":
    train()

```

## 8 Ejecución

Desde la carpeta del proyecto:

```
uv run python train_tabular_as_graph.py
```

Una salida típica debería tener esta forma:

```
Dispositivo: cuda
GPU: NVIDIA ...
Memoria GPU: 8.00 GB
```

```
Dimensiones después de limpieza:
(30000, 12)
```

```
Resumen del grafo
-----
```

```
Nodos: 30000
Aristas: 600000
Variables por nodo: 11
Clases: 2
```

```
Epoch 001 | Loss: ... | Train Acc: ... | Val Acc: ...
```

```
Epoch 010 | Loss: ... | Train Acc: ... | Val Acc: ...
...
Resultado final
-----
Mejor accuracy validación: ...
Accuracy test: ...
```

## 9 Ajustes para una GPU de 8 GB

Configuración inicial recomendada:

```
MAX_NODES = 30_000
K_NEIGHBORS = 10
hidden_channels = 64
```

Si sobra memoria:

```
MAX_NODES = 50_000
K_NEIGHBORS = 10
```

Si aparece error de memoria:

```
MAX_NODES = 15_000
K_NEIGHBORS = 5
hidden_channels = 32
```

La variable que más aumenta el costo es `K_NEIGHBORS`, porque controla el número de aristas.

## 10 Advertencia metodológica

Este ejemplo es **transductivo**.

Eso significa que el grafo se construye usando todos los pacientes disponibles, incluidos entrenamiento, validación y prueba. Sin embargo, las etiquetas de validación y prueba no se usan para actualizar el modelo.

Este esquema es común en muchos ejemplos de GNN, pero debe declararse claramente.

Para un escenario clínico más estricto, se debería construir una versión **inductiva**, donde los pacientes de prueba no participen en el grafo de entrenamiento. En ese caso, se entrena con un grafo de entrenamiento y luego se insertan o evalúan nuevos pacientes con reglas controladas.

## 11 Interpretación biomédica del ejemplo

El grafo construido no representa causalidad. Representa **similitud clínica**.

Por tanto, una arista significa:

Estos dos pacientes se parecen según las variables disponibles.

No significa:

Un paciente causa el estado clínico del otro.

La hipótesis relacional es:

Pacientes clínicamente similares pueden aportar información útil entre sí para predecir  
 ↪ enfermedad cardiovascular.

## 12 Cómo trasladar este esquema a proyectos biomédicos propios

El mismo patrón puede adaptarse así:

| Problema | Nodo                    | Arista                          | Etiqueta           |
|----------|-------------------------|---------------------------------|--------------------|
| EEG      | Canal o región cerebral | Conectividad funcional o causal | Estado neurológico |

| Problema             | Nodo                    | Arista                                     | Etiqueta             |
|----------------------|-------------------------|--------------------------------------------|----------------------|
| Imágenes médicas     | Lesión, región o parche | Proximidad espacial o similitud radiomica  | Tipo de lesión       |
| Pacientes clínicos   | Paciente                | Similitud clínica                          | Diagnóstico o riesgo |
| Señales fisiológicas | Segmento temporal       | Similitud de forma o proximidad temporal   | Evento fisiológico   |
| Variables clínicas   | Variable                | Correlación, dependencia o relación causal | Grupo funcional      |

La parte más importante no es la GNN en sí, sino la definición rigurosa del grafo.

## 13 Conclusión

Este ejemplo muestra cómo pasar de datos tabulares a una estructura de grafo:

```
Fila tabular -> nodo
Similitud entre filas -> arista
Variable objetivo -> etiqueta del nodo
```

La GNN usa las variables de cada paciente y la estructura de similitud para hacer clasificación de nodos.

Desde el punto de vista metodológico, el paso crítico es justificar la regla de conexión. En este documento se usó kNN por similitud clínica, pero en investigación biomédica puede ser más apropiado usar conectividad funcional, distancia anatómica, relaciones fisiológicas o hipótesis causales.

## 14 Referencias breves

- Kaggle. *Cardiovascular Disease dataset*. <https://www.kaggle.com/datasets/sulianova/cardiovascular-disease-dataset>
- PyTorch. *Start Locally*. <https://pytorch.org/get-started/locally/>
- PyTorch Geometric. *torch\_geometric.data.Data*. [https://pytorch-geometric.readthedocs.io/en/2.6.0/generated/torch\\_geometric.data.Data.html](https://pytorch-geometric.readthedocs.io/en/2.6.0/generated/torch_geometric.data.Data.html)
- PyTorch Geometric. *torch\_geometric.nn.conv.GCNConv*. [https://pytorch-geometric.readthedocs.io/en/2.5.1/generated/torch\\_geometric.nn.conv.GCNConv.html](https://pytorch-geometric.readthedocs.io/en/2.5.1/generated/torch_geometric.nn.conv.GCNConv.html)